How Long May a Reader Hold the Lock? A Controlled Study of Coarse Locking, Descriptor Leasing, and Immutable Snapshots in an LSM Engine

V V Ramarathinam Iyer
Department of Database Systems, School of Computer Science and Engineering
Vellore Institute of Technology, Vellore, Tamil Nadu, India
Email: ram.iyer200722@gmail.com

***Abstract*—In this LSM engine, holding a shared lock across a point lookup cost writers at most about 8% below about 40 μs of measured hold, while readers gained throughput. Around 170 μs, writers lost 24–29%; around 1.9 ms, writers lost 64–69% and readers 28–37%. With the old scan-based reader, writers lost over 99%. Four pre-registered campaigns comprising 1,519 trials compared CoarseLock, per-table descriptor leasing (Lease), and immutable reader sets (Snapshot). CoarseLock's unexpected read gain was consistent with two reader admissions per commit under Go's reader-writer lock. A pre-registered intervention supported this explanation: removing the second exclusive acquisition reduced reads per reader per commit from about two to one, removed the detectable low-cost read advantage, reduced both designs' read throughput, and increased read p99. Lease and Snapshot were within the declared $\pm 10\%$ throughput margin in all tested cells starting near four tables, including under compaction. That equivalence had costs and conditions: Lease's commit-wait p99 was 1.4–7.7 times Snapshot's under compaction, and without a writer Lease completed 34% fewer reads at 256 tables and sixteen readers because it leases every table. These findings concern one engine on one host, with closed-loop load and WAL fsync under the exclusive lock.**

***Index Terms*—LSM tree, reader-writer lock, descriptor leasing, immutable snapshots, controlled experiment.**

I. Introduction

A shared lock held across a lookup is an easy way to keep a storage file alive, but its cost depends on how long the lookup takes. An LSM engine buffers updates in memory and writes sorted runs to disk [1], [2]. Readers need a stable view of those runs while flushes and compactions replace them. Keeping the manager's read lock throughout a lookup prevents concurrent removal with little lifetime-management code. It also makes every exclusive lock acquisition wait for the entire lookup, including file I/O.

Production engines separate these concerns. LevelDB, RocksDB, and Pebble capture references that let readers continue after releasing a short coordination lock [3]–[5]. Advice against holding a shared lock across I/O is therefore easy to motivate. It is less often quantified: we did not find a controlled measurement that answers how long a reader may hold such a lock before replacing the design becomes worthwhile. A fast cached read and a millisecond read need not present the same trade-off, especially when writers already spend substantial time inside the exclusive lock.

We study that trade-off in a small Go LSM engine with three lifetime designs. CoarseLock holds the manager read lock across the lookup. Lease captures a lease on each active table reader, and Snapshot pins one immutable reader-set descriptor; both release the manager lock before reading files. These are our implementations, not reproductions or benchmarks of the production systems. We vary a delay immediately before each SSTable block read to make read cost a controlled variable, then test the limits of the resulting comparison on new data.

The study asks three questions. How does CoarseLock's write penalty change as the lock-held lookup gets longer, and what happens to reads? Why can holding the lock produce more reads, and does changing reader admission alter that result? Finally, does Lease match Snapshot within a declared throughput margin when reader sets are published, files are retired by compaction, or the active table count grows?

The answer is conditional. At measured holds below about 40 μs, CoarseLock's point-estimate write losses stayed below about 8% and reads increased. At millisecond holds, both outcomes deteriorated. The unpredicted low-cost read gain led us to an admission explanation and a separate pre-registered intervention, which supported it while exposing a read/write trade-off. Lease matched Snapshot's throughput under compaction near the original table count, but differed in commit-wait tails and lost read throughput as table count grew without a writer.

This paper makes the following contributions:

- A controlled, pre-registered dose-response comparison that relates injected delay to measured lock hold, rather than treating all file reads as equally expensive.
- Measurements of the increasing write penalty and the unpredicted read-throughput crossover, with paired confidence intervals (CIs).
- An exploratory admission explanation tested by a separately pre-registered intervention that changed the number of exclusive acquisitions per commit.

- Bounded evidence that Lease and Snapshot meet a $\pm 10\%$ throughput criterion near four initial tables, including under compaction, but not for read-only loads at large table counts or for publication and commit-wait tails. The study also corrects an earlier attribution of churn cost to snapshot lifecycle.

The paper describes the engine and design, presents the four campaigns, and then discusses their implications, limitations, and relationship to prior work.

II. Background

A. The lifetime problem and production designs

Stable file references let a reader outlive a short coordination critical section. In LevelDB's `db/db_impl.cc`, a read takes references on the memtables and current version under the mutex, releases the mutex while reading files and memtables, and later releases those references [3]. The write path also releases the mutex around log append and synchronization, with a leader batching queued writers. RocksDB readers take a reference on a SuperVersion through `GetAndRefSuperVersion` [4]. Pebble readers take a reference on a refcounted read state under a short `RLock` [5].

These designs motivate the lifetime question without making the systems interchangeable. The three designs here are small implementations within one engine. Its transaction validation, WAL synchronization, file format, and publication path remain part of every comparison. In particular, a reader-side lifetime change need not shorten the writer's own critical section.

B. Reader-writer admission

A reader-writer lock governs who goes next as well as who may overlap. The readers-writers problem permits concurrent readers while excluding writers [12], but different admission policies can produce different throughput under the same critical-section durations. Go's `RWMutex` blocks new readers once a writer waits and releases queued readers on writer unlock [18]. A continually waiting writer can therefore alternate with admitted readers. Phase-fair locks explicitly address the ordering of reader and writer phases [14]; we use that literature to frame admission, without formally classifying Go's implementation as phase-fair.

C. Earlier measurements

A defective reader motivated the present study but supplies no pooled evidence for it. That reader scanned from the target block toward the data end, used about four unbuffered syscalls per record, and did not stop after passing the target key. A 960-trial WSL2 campaign comparing CoarseLock and Snapshot and a Windows campaign stopped by its stopping rule at 60 of 330 planned trials, with 18 complete triplets, used this path. They reported Snapshot completing 3.3–18.5 times CoarseLock's writes and churn throughput ratios of 0.63–0.74 at $r \leq 4$. We retain the defective path only as the legacy scan operating point. The earlier campaigns motivated the controlled delay, were not pooled with these data, and carry no inference into this paper. This study supersedes their claims, including the withdrawn $\pm 5\%$ equivalence claim and any earlier claim about Lease under churn.

III. The Engine and Three Designs

A. Storage and point reads

The engine uses the same storage path in all three designs. Its memtable is a skip list [11] with promotion probability 0.5 and maximum height 32. Internal keys append an eight-byte big-endian, bitwise-inverted encoding of $(\mathrm{seq} \ll 8) \,|\, \mathrm{delete}$ to the user key. This gives 56 effective sequence bits and a low-bit tombstone indicator, with newer versions sorting first within each user key. Frozen memtables flush into SSTables.

Data blocks use a 4 KiB flush threshold, which a whole record may exceed. A flat sparse index contains first keys and offsets, and a footer locates the index. The corrected point reader first checks the cached user-key range, binary-searches the index, and issues one `ReadAt` per visited block. It stops after finding a visible version or passing the user key. Versions may straddle blocks, so some reads visit multiple blocks. Open-time range-metadata reads bypass the injected delay, and there is no application block cache.

B. Three ways to keep readers alive

The designs differ in what protects a file reader after the memory search. All three search active and frozen memory while holding `Mu.RLock`. CoarseLock retains this lock through SSTable lookup and I/O. This gives file lifetime protection directly: exclusive publication and retirement cannot proceed until readers leave.

Lease moves file access outside the manager lock by protecting individual readers. While holding the read lock, it copies table-reader pointers into reusable scratch storage and takes an atomic lease on every active table. It then searches outside `Mu` and releases all leases afterwards. Shutdown closes admission and drains active and retired leases. A timeout returns an error while preserving descriptors that remain leased. The per-table work is central to the later table-count experiment, even when range checks make only one table relevant to a lookup.

Snapshot instead pins one immutable reader-set descriptor under the short manager read lock and `snapshotMu`. It searches outside both locks and reacquires `snapshotMu` to release the pin. Publication allocates a new set, assigns `currentSnapshot` under the mutexes, and retires the previous set. Once a retired set drains, it releases its constituent references; eligible closure and unlinking run outside both mutexes. Publication is mutex-coordinated, not a lock-free pointer swap. Table I summarizes these audited implementation differences.

C. The commit path

Every successful default-path commit takes the exclusive manager lock twice, and the first hold includes both conflict validation and WAL fsync. `Transaction.Commit` acquires `Mu.Lock`, validates

TABLE I
LIFETIME DESIGNS WITH THE DEFAULT COMMIT PATH.

Property	CoarseLock	Lease	Snapshot
Memory-tier search under `Mu.RLock`	yes	yes	yes
Reader SSTable I/O under `Mu.RLock`	yes	no	no
Disk-reader lifetime	lock	table leases	set pin
Per-read lifetime synchronization	`Mu`	atomic leases	`snapshotMu`
New set descriptor on publication	no	no	yes
Conflict check and WAL sync under `Mu.Lock`	yes	yes	yes
Exclusive acquisitions per successful commit	2	2	2

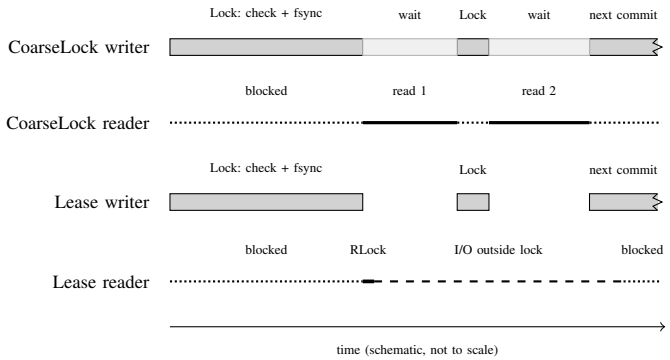


Fig. 1. One commit cycle under CoarseLock and Lease (schematic, not a measured trace). Under CoarseLock the writer waits for each read and each reader completes one read per exclusive acquisition. Under Lease the reader's I/O is outside the lock and can outlast the short second acquisition. CoarseLock: approximately two reads per reader per commit; Lease: fewer.

conflicts through `latestValueLocked`, appends and synchronizes the WAL, updates memory, and unlocks. It then calls `checkAndTriggerFlush`, which reacquires the exclusive lock even when no flush is needed. Only the first acquisition's wait is instrumented.

Conflict validation can read SSTable blocks under the exclusive lock in every design. The injected delay therefore reaches the writer as well as readers. Lease and Snapshot shorten the reader's hold; neither changes the exclusive validation or fsync. The two acquisitions also create two opportunities to admit queued readers. Figure 1 sketches the admission explanation developed after the first campaign.

IV. EXPERIMENTAL DESIGN

A. Host and measurement

The campaigns hold the host and measurement conventions fixed while changing the factors needed for each question. The host is an AMD Ryzen 7 7445HS with six cores and twelve logical CPUs, 15.2 GiB RAM, and a Samsung MZVL8512HFLU NVMe SSD. It runs Windows 11 Home build 26200 on NTFS; the runner uses Go 1.26.3 and `GOMAXPROCS=12`. Timing uses QueryPerformanceCounter. The operator reported AC power and retained Defender real-time protection; the limits of the power record are discussed in Section VII-B.

Except where noted for table count, fixtures contain 10,000 keys in four disjoint SSTables of 2,500 keys, with 128-byte values. Readers issue uniform, closed-loop lookups with target hit ratio 0.95. Trials have 2 s reader-only warmup, a 10 s measurement window, and 200 ms cooldown, with synchronization instrumentation enabled. WAL fsync is enabled during measurement and disabled while constructing fixtures.

TABLE II
FIRST-CAMPAIGN CELLS. THE LEGACY SCAN HAS NO ADDED DELAY.

Workload	Readers	Injected delay	Cells
Mixed	4, 16	0, 20, 200 μs, 2 ms, legacy scan	10
Publication	16	0, 200 μs, 2 ms, legacy scan	4
Churn	1, 4, 16	0	3

We write r for reader count, d for injected delay, W and R for write and read throughput, $S = W_{\mathrm{CL}}/W_{\mathrm{L}}$, $q = R/(rW)$ for reads per reader per commit, $T = 1/W$ for a commit cycle, and w for mean first-acquisition commit wait. Subscripts CL, L, and S denote CoarseLock, Lease, and Snapshot, respectively. In the table-count experiment, N denotes initial SSTable count; writer count is stated in words to keep w reserved for wait duration.

B. Workloads and controlled read cost

The first campaign separates a sustained writer from scheduled publication and frequent flushing. Table II lists its cells. The Mixed workload has a free-running writer and a 1 MiB memtable. The Publication workload has no writer: twenty pre-frozen memtables are released for physical publication every 0.5 s, from 0.25 to 9.75 s. The Churn workload uses a 16 KiB memtable and one free-running writer to exercise frequent reader-set publication.

The delay varies lookup cost without changing the file format or lifetime design. It executes immediately before each point-read `ReadAt`, including those in conflict validation. Below 1 ms it uses a CPU-yielding spin; the 2 ms dose sleeps and then spins for the final 0.5 ms. This is a controlled proxy for read cost, not a model of a storage device. An earlier single-goroutine probe measured approximately 9, 35, and 215 μs and 2.25 ms per read at the four doses. Those realized single-read costs differ from concurrent lock holds, which we report separately. The legacy scan is a separate historical point and is excluded from the ordered dose test.

C. What the trials exercised

The four campaigns deliberately extend the lifetime activity covered by the initial measurements. Table III counts diagnostics from trial envelopes, including drain. Mixed includes the nine anchor trials and had at most one flush per trial. Publication and Churn exercised reader-set publication, but the first campaign and the intervention ran no compactions because the compaction trigger was disabled. Since Lease and Snapshot differ most when reader sets change, throughput equivalence in a mostly stable set did not establish equivalence under file retirement. The compaction campaign was run to close that gap.

The intervention isolates acquisition count in eight Mixed-shaped cells: two reader counts and four non-legacy doses.

TABLE III
WHAT THE TRIALS EXERCISED. FLUSH COUNTS INCLUDE DRAIN; COMPACTIONS COUNT COMPLETED WORK, NOT NO-OP TIMER CALLS.

Workload/campaign	Trials	Flushes/trial Mean	Max	Total compactions
Mixed	309	0.55	1	0
Publication	120	20.00	20	0
Churn	90	60.19	77	0
Intervention	160	0.66	1	0
Compaction	360	36.22	87	5364
Table-count	480	0.50	1	0

It compares the default two-acquisition commit with a one-acquisition commit below the flush threshold in CoarseLock and Lease. Five rounds share a seed across each four-run cell-round unit, giving 160 trials. The default condition is rerun in the same session rather than borrowed from the first campaign. Snapshot is omitted. Section V-D gives the predictions and exact change.

The compaction campaign uses Churn-shaped cells with $r \in \{4, 16\}$, $d \in \{0, 200, 2000\}\,\mu$s, and compaction on or off. On uses a fixed 250 ms timer that merges all active tables into one; calls with at most one table do no work. Twelve cells, three designs, and ten rounds give 360 trials. On and off share seeds within the same reader-count, delay, and round. The implementation has a disclosed asymmetry: CoarseLock and Lease open compaction inputs under the exclusive lock, whereas Snapshot opens them outside all locks. This is not an inherent requirement of leasing.

The table-count campaign removes overlapping-table search as a factor by constructing disjoint ranges. It crosses $N \in \{4, 16, 64, 256\}$, $r \in \{4, 16\}$, and zero or one free-running writer. Sixteen cells, three designs, and ten rounds give 480 trials. The fixed 10,240-key space yields 2,560, 640, 160, or 40 keys per initial table. Delay is zero, the memtable threshold is 1 MiB, and no compaction is scheduled. A writer may still flush, so the analysis retains initial and final counts. All designs perform up to N in-memory range comparisons, under the manager lock only in CoarseLock.

Separate lifecycle microbenchmarks in the table-count results measure capture/release at one and sixteen goroutines and uncontended publication from N to $N+1$ tables. They exclude table search, memory-tier lookup, and disk I/O. Lease's test helper mirrors its scratch-copy and lease path; Snapshot uses its production pin/release path. The publication benchmark includes immediate retirement of an unpinned old Snapshot set; Lease models amortized append with spare slice capacity. Scratch is primed before timing. We report the arithmetic mean of three benchmark counts, separately from campaign throughput, without confirmatory tests.

D. Pairing, measures, and analysis

Pairing controls operation streams, while randomized interleaving limits order effects. The first campaign uses ten matched triplets per cell and three anchor triplets. Designs share the seed and operation-index streams, although faster designs can complete longer prefixes. Ten rounds randomize cell order and cycle design order through six permutations. The compaction and table-count campaigns also use ten rounds, all three designs, and the same equivalence margin and paired statistics. Seed derivation and execution provenance are in the Artifact Appendix.

Throughput counts completions inside the measurement window. The first, compaction, and table-count campaigns divide by the measured window; the intervention uses the fixed 10 s denominator specified in its plan. Late completions are recorded separately. Latency histograms retain every in-window completion without sample drops. Synchronization counters reset at window start and include post-window drain; flush diagnostics likewise include drain, so their counts are not strictly in-window event counts.

The first campaign's H1 tests a decreasing S across the four non-legacy doses using the one-sided exact Page test [23], blocked by triplet. H2 uses two one-sided tests (TOST) [24], [27] on paired log Lease/Snapshot ratios. It reports exponentiated 90% paired-t CIs and applies Holm correction [25] separately to ten write, fourteen read, and four publication lock-wait p99 tests. An adjusted $p < 0.05$ establishes equivalence; a CI wholly beyond a margin is classified not equivalent, and other outcomes are inconclusive. H3 reports two-sided 95% paired-t CIs under Churn without a directional prediction. Descriptive ratios use 95% CIs. Percentile bootstrap sensitivity [26] uses 10,000 resamples and fixed seeds; it does not replace the planned tests. Cells are not pooled, and no planned write-zero adjustment was needed.

The margin is fixed before data at $\pm\ln(1/0.90)$, or ratios $[0.90, 1/0.90]$, described here as $\pm 10\%$ on the multiplicative scale. It is the author's judgment of a difference small relative to the complexity difference between designs, chosen so ten pairs could resolve it. It is not derived from a production requirement. The pre-registration used a pilot standard deviation of paired log ratio near 0.06, giving an expected 90% CI half-width near 0.035 at ten pairs. Full CIs let readers apply a tighter margin.

The new campaigns preserve this method while defining their own families. The compaction primary hypothesis (H-C) predicts equivalence in all six on cells for each of writes and reads, with separate six-test Holm families; off cells form separate reference families. The table-count trend hypothesis (H-T1) uses four Page tests, one per reader-count and writer-count combination, with Holm across those four. Its equivalence families (H-T2) contain sixteen read and eight applicable write tests. H-T2 predicts equivalence at four tables for each applicable outcome and non-equivalence for reads at 256 tables without a writer. The intervention has five paired units per cell and 95% CIs without multiplicity correction, as specified before data.

Pre-registration distinguishes hypothesis generation from testing [28]. Each protocol was frozen in git before non-smoke trials. The original read gain, its admission explanation, the commit-cycle check, the $\pm 5\%$ margin sensitivity, and

TABLE IV
MIXED WORKLOAD: MEASURED COARSELOCK HOLD AND PERCENTAGE CHANGE RELATIVE TO LEASE, USING PAIRED GEOMETRIC RATIOS. THESE ARE POINT ESTIMATES; TABLE V GIVES CIS.

r	Delay	Hold (μs)	Writes	Reads
4	0	13.6	-1.2%	+19.3%
4	20 μs	33.6	-7.4%	+45.2%
4	200 μs	163	-24.2%	+18.7%
4	2 ms	1,885	-63.9%	-27.5%
4	legacy scan	124,853	-99.6%	-51.1%
16	0	21.2	-6.3%	+9.7%
16	20 μs	39.3	-5.6%	+43.2%
16	200 μs	173	-29.5%	+9.3%
16	2 ms	1,915	-68.9%	-37.5%
16	legacy scan	430,130	-99.8%	-47.8%

interpretations of compaction tails and table-count effects are exploratory. The intervention and the new campaigns test predictions on new data; they do not retroactively make the initial explanation confirmatory. Smoke exposure and the aborted start are disclosed in the threats and Artifact Appendix.

V. RESULTS

Integrity. All 519 first-campaign trials succeeded on the first attempt, with no exclusions; so did the 160 intervention, 360 compaction, and 480 table-count trials. There were no retries or dropped paired units across the 1,519 trials. Pre-registered first-campaign anchor drift was $D = 0.979$, $|\ln D| = 0.021$, below the 0.10 threshold. The later ten-round campaigns have descriptive round trends rather than anchor-based drift tests.

A. How long is too long?

CoarseLock cost writers little at short measured holds, but its penalty grew rapidly beyond that range. Figure 2 plots the paired ratios against measured mean read-lock hold, and Table IV translates them into throughput changes. At about 34 μs of measured hold and four readers, CoarseLock completed 7.4% fewer writes than Lease (ratio 0.926, 95% CI $[0.881, 0.974]$) and 45% more reads. Once the hold reached about 1.9 ms the picture reversed: writes fell by 64% and reads by 28%.

H1 supported a decreasing write-ratio trend at both reader counts: Page's $L = 294$ and exact one-sided $p = 7.86 \times 10^{-9}$, with ten paired blocks each. That trend was not strictly monotonic in the point estimates. At sixteen readers, S rose from 0.937 at zero delay to 0.944 at 20 μs; their mean ranks were 3.4 and 3.6. The result does not distinguish every adjacent dose. Around 163–173 μs of measured hold, write losses were 24–29%; at about 1.9 ms, they were 64–69%. The legacy scan produced losses above 99%, at mean holds of about 125 and 430 ms.

Below about 40 μs, the largest observed point-estimate loss was 7.4%. This is not a confidence bound or a universal threshold: at 20 μs injected delay, the 95% ratio intervals were $[0.881, 0.974]$ at four readers and $[0.887, 1.004]$ at sixteen. The intervals allow larger losses, and the latter includes one. Figure 3 and Table V retain the rates and uncertainty behind this summary.

Writes also fell in Lease and Snapshot as delay increased. The writer's conflict check reads delayed blocks while holding the exclusive lock in every design. Lease's mean commit cycle at four readers rose from 1.07 ms at zero delay to 2.79 ms at 2 ms delay. Thus the denominator of S also degraded: neither alternative removes the writer's own delayed reads.

B. Read throughput rose before it fell

Readers benefited from CoarseLock through the 200 μs dose before losing throughput at 2 ms. This crossover was not predicted. At four readers, CoarseLock/Lease read ratios ranged from 1.19 to 1.45 through 200 μs, then fell to 0.725 at 2 ms; at sixteen readers the corresponding ranges were 1.09–1.43 and 0.625. Figure 4 shows the three designs. The larger write loss and eventual read loss are consistent with a longer shared hold, but the earlier read gain needs an admission explanation.

Median latency alone could give a misleading impression of that gain. At four readers and zero delay, the geometric mean trial read p50 was 45 μs under CoarseLock and 808 μs under Lease, while p99 was 2.27 and 2.31 ms. Counters and percentiles suggested cycles that mixed a fast read with a read blocked behind an approximately 1 ms commit. We inferred bimodality without a full reported histogram; we did not establish a latency distribution from those percentiles. Throughput and p99 give necessary context for the median.

C. Why CoarseLock read more (exploratory)

The exploratory explanation is that CoarseLock makes each exclusive acquisition wait for an admitted read to finish, coupling reads to commits. Post-data source review identified the two exclusive acquisitions and Go's admission behavior. With a continually waiting writer and approximately one read per reader admission, two acquisitions predict

$$R_{\mathrm{CL}} \simeq 2rW_{\mathrm{CL}}. \tag{1}$$

Across individual trials in the eleven non-legacy Mixed and Churn cells, q ranged from 2.002 to 2.015 under CoarseLock. The legacy range was 1.98–2.29, with only about 26 commits per trial at four readers and six at sixteen. The non-legacy coupling remained near two despite changing delay and reader count.

Lease and Snapshot do their file I/O outside the manager lock. A reader still doing I/O can miss the short second acquisition's admission opportunity and then block behind the next commit. Their cell mean q fell from about 1.7 at zero delay to about 1.3 at 20 and 200 μs, and about one at 2 ms (Table VI). The schematic in Fig. 1 illustrates this account; it is not a measured trace or a guarantee that every reader completes exactly one read per admission.

A descriptive cycle check asked whether two waits could account for CoarseLock's longer cycle:

$$\widehat{T}_{\mathrm{CL}} = T_{\mathrm{L}} + 2w_{\mathrm{CL}}. \tag{2}$$

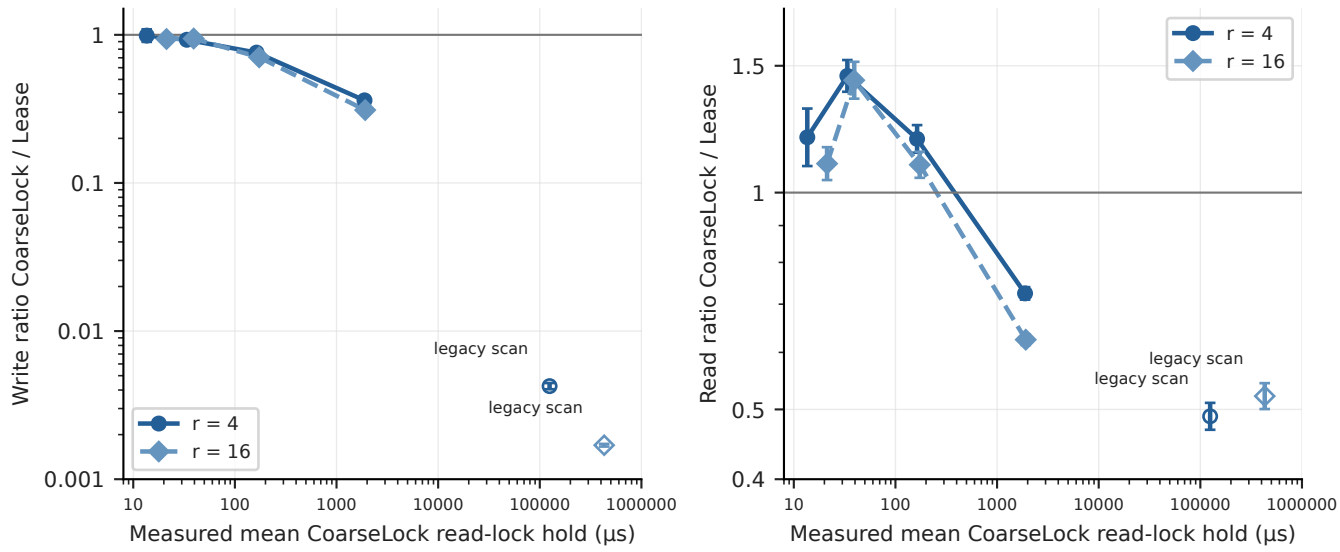


Fig. 2. Mixed workload: paired CoarseLock/Lease ratios against CoarseLock's arithmetic mean trial read-lock hold. Bars are 95% paired log-t CIs. Lines connect the four injected doses; hollow legacy scan points are unconnected. Measured hold is descriptive, not a separately randomized treatment.

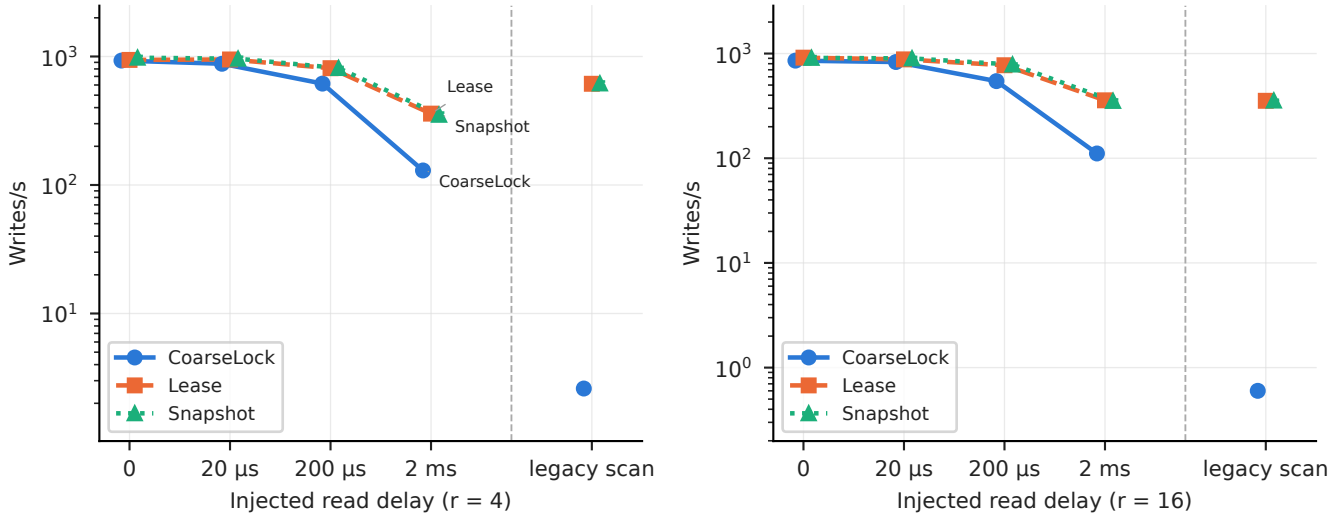


Fig. 3. Mixed workload writes, ten triplets per cell. Points are per-design geometric means; bars are exponentiated 95% log-t CIs. Most intervals are smaller than the markers. The legacy scan is an isolated category.

We averaged per-trial cycles, rather than taking reciprocals of aggregate rates. Across the ten Mixed cells, prediction error ranged from -13.8% to $+10.6\%$. Only the first wait was measured, so this check assumes an equal second wait. Agreement is a consistency check, not validation of a causal model. The next campaign directly changed the second acquisition below threshold.

D. Testing the explanation by intervention

The intervention supported the acquisition-count explanation while showing that the second acquisition also benefits readers. An experiment-only flag, false by default, moves the flush-threshold check into the first exclusive hold. Below threshold it skips the second acquisition, giving the one-acquisition commit. At or above threshold, the existing path still reacquires the lock and rechecks before rotation and flush. Validation, WAL work, and other lock sites remain unchanged. The two-acquisition commit is the default condition.

The five-round design pairs both conditions in CoarseLock and Lease within each cell. P1 predicts CoarseLock's arithmetic mean q in $[1.90, 2.10]$ under two acquisitions and $[0.90, 1.10]$ under one, in all eight cells. P2 predicts that removing the acquisition lowers CoarseLock's read ratio relative to Lease at delays up to 200 μs: the 95% paired-t CI of the

TABLE V

MIXED WORKLOAD, TEN TRIPLETS PER CELL. RATES ARE GEOMETRIC MEANS (OPERATIONS/S); HOLD IS THE ARITHMETIC MEAN OF COARSELOCK'S TRIAL MEAN READ CRITICAL SECTIONS. PAIRED RATIOS HAVE 95% t CIS.

r	Delay	Hold μs	Writes/s CoarseLock	Lease	Snapshot	S [95% CI]	Reads/s CoarseLock	Lease	Snapshot	R_{CL}/R_L [95% CI]
4	0	13.6	929	940	978	0.988 [0.896, 1.089]	7,468	6,258	6,482	1.193 [1.089, 1.308]
4	20 μs	33.6	878	948	961	0.926 [0.881, 0.974]	7,035	4,844	4,909	1.452 [1.380, 1.528]
4	200 μs	163	615	811	813	0.758 [0.725, 0.793]	4,932	4,154	4,161	1.187 [1.136, 1.241]
4	2 ms	1,885	130	359	352	0.361 [0.354, 0.368]	1,039	1,434	1,408	0.725 [0.710, 0.739]
4	legacy scan	124,853	2.6	613	617	0.004250 [0.004049, 0.004461]	21.2	43.3	43.3	0.489 [0.468, 0.511]
16	0	21.2	856	914	909	0.937 [0.884, 0.993]	27,538	25,101	24,972	1.097 [1.041, 1.156]
16	20 μs	39.3	830	880	899	0.944 [0.887, 1.004]	26,668	18,620	18,966	1.432 [1.350, 1.519]
16	200 μs	173	545	773	785	0.705 [0.676, 0.736]	17,507	16,017	16,256	1.093 [1.048, 1.140]
16	2 ms	1,915	111	358	352	0.311 [0.305, 0.316]	3,557	5,693	5,617	0.625 [0.614, 0.636]
16	legacy scan	430,130	0.6	354	355	0.001696 [0.001661, 0.001732]	21.2	40.6	40.7	0.522 [0.501, 0.544]

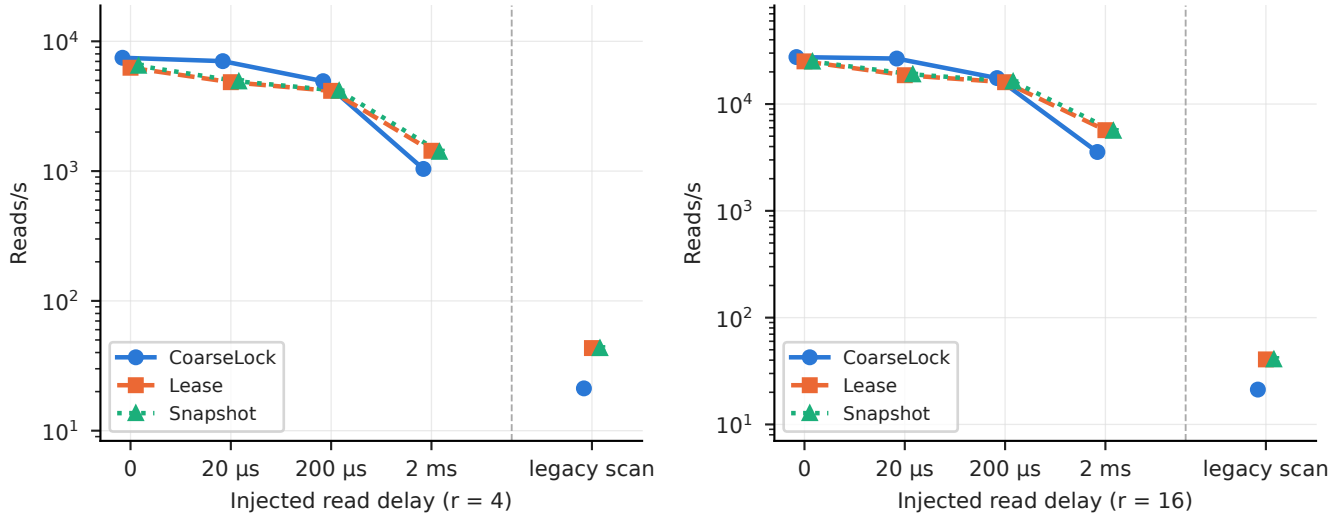


Fig. 4. Mixed workload reads, ten triplets per cell. Points are geometric means; bars are exponentiated 95% log-t CIs. CoarseLock's low-dose read gain was unpredicted.

ratio of read ratios must lie wholly below one in six cells. These are unadjusted five-pair comparisons, of descriptive strength as specified in the protocol. P3 predicts cycle error within $\pm 15\%$ for both conditions, using two waits or one as appropriate, and a CoarseLock write improvement with its 95% CI above one at 200 μs and 2 ms.

All predictions met their criteria. P1 held in all sixteen condition-cells: individual CoarseLock trial q ranged from 2.002–2.013 under two acquisitions and 1.001–1.006 under one. P2 held in all six applicable cells. For example, at four readers and 20 μs, the ratio of read ratios was 0.659, 95% CI $[0.595, 0.730]$. Under one acquisition, CoarseLock's read-ratio CIs included one at the two lowest doses and fell below one at 200 μs (Fig. 5 and Table VII). The loss of a detectable low-cost advantage and its reversal at 200 μs were not separately predicted crossings. P3's cycle errors were -3.7% to $+7.5\%$ under two acquisitions and -2.0% to $+8.4\%$ under one; all four predicted write-improvement CIs exceeded one.

Unpredicted effects make this a trade-off. Removing the second wait reduced CoarseLock's high-dose write penalty, but a penalty remained. Through 200 μs, both designs then had $q \simeq 1$, making the read ratio close to S. Absolute read throughput fell in both designs: one-acquisition/two-acquisition ratios were 0.507–0.763 for CoarseLock and 0.608–0.850 for Lease. Lease's write ratios were 0.970–1.044, with every CI including one. Its q under one acquisition fell further to 0.857–0.858 at 2 ms, an effect we did not investigate.

Read tails also worsened. Geometric means of paired trial-p99 ratios were 1.330–2.049 for CoarseLock and 1.578–2.125 for Lease. These descriptive values come from raw trial percentiles because the intervention results JSON omits latency percentiles; no CI or additional test was introduced. The rerun baseline reproduced the admission pattern but not identical ratios: at sixteen readers and 20 μs, S was 0.855 versus 0.944 in the first campaign. We did not pool the cohorts. The intervention supports this explanation in this engine and workload. The second acquisition was not simply waste: it gave readers another admission, so removing it exchanged reader throughput and tail latency for CoarseLock writes.

TABLE VI

EXPLORATORY ADMISSION AND CYCLE CHECK: ARITHMETIC MEANS OVER TEN TRIALS PER CELL. CYCLES ARE MS; ERROR IS $100(\widehat{T}/T - 1)$. CHURN REPORTS ADMISSION RATIOS ONLY.

Workload	r	Delay	q_{CL}	q_{L}	q_{S}	T_{CL}	$\widehat{T}_{\mathrm{CL}}$	Error (%)
Mixed	4	0	2.009	1.664	1.657	1.077	1.130	4.9
Mixed	4	20 μs	2.003	1.278	1.277	1.140	1.175	3.1
Mixed	4	200 μs	2.004	1.280	1.280	1.627	1.738	6.8
Mixed	4	2 ms	2.003	0.998	1.001	7.712	7.432	-3.6
Mixed	4	legacy scan	2.032	0.018	0.018	384.3	363.6	-5.4
Mixed	16	0	2.010	1.717	1.718	1.169	1.249	6.8
Mixed	16	20 μs	2.008	1.323	1.318	1.206	1.334	10.6
Mixed	16	200 μs	2.006	1.295	1.294	1.835	1.978	7.8
Mixed	16	2 ms	2.003	0.995	0.996	9.008	8.726	-3.1
Mixed	16	legacy scan	2.209	0.007	0.007	1,666.7	1,437.3	-13.8
Churn	1	0	2.009	1.080	1.081	–	–	–
Churn	4	0	2.009	1.100	1.099	–	–	–
Churn	16	0	2.013	1.104	1.101	–	–	–

TABLE VII

INTERVENTION. q IS ARITHMETIC; THROUGHPUT RATIOS ARE PAIRED GEOMETRIC MEANS, WITH 95% PAIRED LOG-t CIS WHERE SHOWN. COLUMN NUMBERS 2 AND 1 DENOTE TWO-ACQUISITION AND ONE-ACQUISITION COMMITS.

		q_{CL}		q_{L}		$W_{\mathrm{CL},1}/W_{\mathrm{CL},2}$	S		$R_{\mathrm{CL}}/R_{\mathrm{L}}$ [95% CI]	
r	Delay	2	1	2	1	[95% CI]	2	1	2	1
4	0	2.008	1.004	1.655	1.004	1.014 [0.943, 1.091]	0.965	0.963	1.170 [1.066, 1.285]	0.963 [0.897, 1.033]
4	20 μs	2.003	1.002	1.281	1.002	1.032 [0.928, 1.147]	0.946	0.974	1.479 [1.363, 1.604]	0.975 [0.882, 1.077]
4	200 μs	2.003	1.002	1.281	1.002	1.191 [1.131, 1.253]	0.731	0.864	1.144 [1.062, 1.232]	0.864 [0.758, 0.985]
4	2 ms	2.003	1.001	0.998	0.857	1.468 [1.429, 1.508]	0.356	0.539	0.715 [0.706, 0.726]	0.630 [0.592, 0.671]
16	0	2.010	1.003	1.722	1.003	1.091 [1.050, 1.134]	0.945	0.987	1.103 [1.064, 1.142]	0.987 [0.895, 1.089]
16	20 μs	2.008	1.003	1.320	1.003	1.123 [0.992, 1.271]	0.855	0.983	1.301 [1.174, 1.443]	0.983 [0.897, 1.077]
16	200 μs	2.006	1.001	1.290	1.001	1.274 [1.175, 1.382]	0.662	0.848	1.030 [0.993, 1.068]	0.848 [0.822, 0.875]
16	2 ms	2.003	1.001	0.997	0.858	1.528 [1.507, 1.548]	0.310	0.479	0.623 [0.603, 0.643]	0.559 [0.553, 0.566]

E. Lease versus Snapshot

Lease and Snapshot met the pre-registered throughput equivalence criterion in every Mixed and Publication cell. All ten write and fourteen read tests passed after Holm correction (Fig. 6). All corresponding bootstrap intervals also lay inside the declared margin, without replacing TOST by a bootstrap test. Equivalence does not imply equality: Publication at zero delay had read ratio 0.970, 90% CI $[0.962, 0.978]$, a detectable roughly 3% difference within the margin.

A post-hoc tighter-margin check asks only whether each existing 90% CI fits inside $[0.95, 1/0.95]$, or approximately $[0.95, 1.0526]$. It finds 6 of ten write cells and 11 of fourteen read cells inside. Every interval outside that tighter band occurs at $d \leq 20$ μs. This is interval sensitivity, not a new Holm-adjusted equivalence result. Publication lock wait requires a separate conclusion below; Sections V-H and V-I test the throughput result under retirement and more tables.

F. Publication

Scheduled publications met their in-window targets except under CoarseLock with the legacy scan. At zero delay, 200 μs, and 2 ms, every design published all twenty tables in every trial. Under the legacy scan, CoarseLock published a median 10.5, range 9–11, whereas Lease and Snapshot still completed twenty. Figure 7 and Table VIII separate the completion count from lock-wait tails.

CoarseLock's geometric mean trial publication lock-wait p99 rose from 247 μs at zero delay to 3.89 ms at 2 ms and 1.05 s under the legacy scan; the legacy arithmetic mean was 1.06 s. Lease and Snapshot stayed below 42 μs in every cell, but their waits were not established as equivalent. At zero delay, Lease/Snapshot was 3.11, 90% CI $[1.54, 6.30]$, classified not equivalent; the other cells were inconclusive (Table IX). A p99 over twenty events is close to a maximum and is noisy. Throughput equivalence therefore does not establish publication-wait equivalence.

G. Churn

The Churn results do not support attributing the read deficit relative to CoarseLock to Snapshot's descriptor lifecycle. Mean measured flushes ranged from 38.8 to 73.2 per trial, including drain. All Lease/Snapshot 95% paired-t CIs included one, but bootstrap intervals at one and sixteen readers narrowly excluded one for both outcomes; for example, the one-reader read CI was $[0.909, 0.997]$ (Table X). Small Snapshot-favoring differences remain possible.

CoarseLock/Snapshot read ratios were 1.66, 1.53, and 1.29 across the reader counts. The deficit relative to CoarseLock also appeared in Lease, which does not allocate reader-set

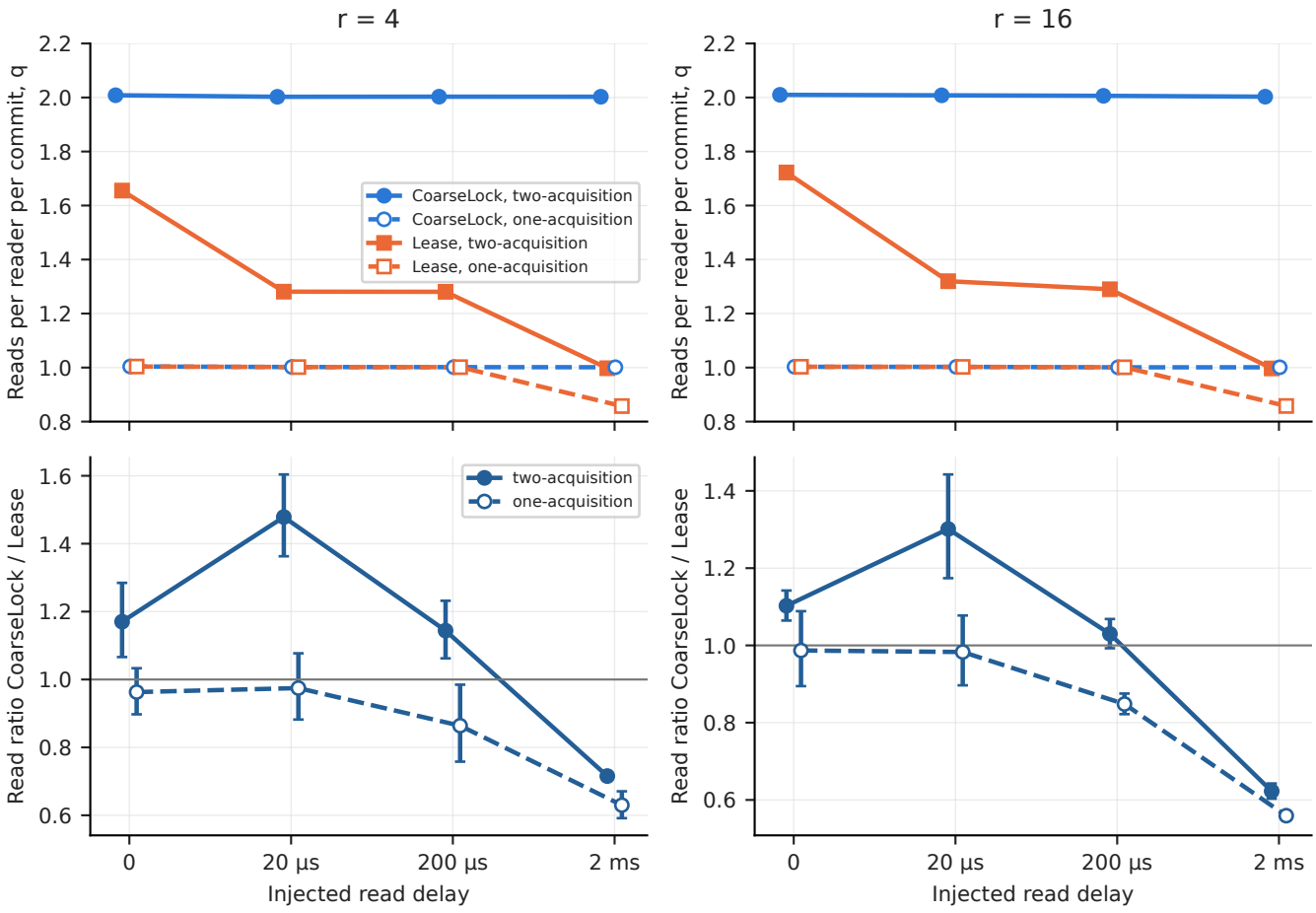


Fig. 5. Intervention, five paired four-run units per cell. Top: arithmetic mean reads per reader per commit. Bottom: paired geometric CoarseLock/Lease read ratios with 95% log-t CIs. Solid filled markers denote two-acquisition commits; dashed hollow markers denote one-acquisition commits below threshold.

TABLE VIII
PUBLICATION WORKLOAD. RATES AND TRIAL LOCK-WAIT P99 (μS) ARE GEOMETRIC MEANS. PUBLICATION COUNTS ARE MEDIAN [MIN, MAX], OUT OF TWENTY.

	Reads/s			In-window publications			Lock-wait p99 (μs)		
Delay	CoarseLock	Lease	Snapshot	CoarseLock	Lease	Snapshot	CoarseLock	Lease	Snapshot
0	318,567	304,721	314,201	20 [20, 20]	20 [20, 20]	20 [20, 20]	247	41.60	13.37
200 μs	75,018	74,678	74,844	20 [20, 20]	20 [20, 20]	20 [20, 20]	450	19.33	12.10
2 ms	8,437	8,445	8,450	20 [20, 20]	20 [20, 20]	20 [20, 20]	3,886	1.51	2.83
legacy scan	27.6	42.7	42.7	10.5 [9, 11]	20 [20, 20]	20 [20, 20]	1,054,655	1.88	1.74

TABLE IX
PUBLICATION LOCK-WAIT P99: PAIRED LEASE/SNAPSHOT RATIO, 90% t CI, AND PLANNED CLASSIFICATION.

Delay	Ratio [90% CI]	Classification
0	3.11 [1.54, 6.30]	not equivalent
200 μs	1.60 [0.51, 4.98]	inconclusive
2 ms	0.53 [0.22, 1.29]	inconclusive
legacy scan	1.08 [0.64, 1.82]	inconclusive

descriptors. The earlier snapshot-lifecycle attribution therefore fails this control, and these data do not show that Lease avoids a churn penalty. Admission ratios near 2.01 under CoarseLock and about 1.1 under the alternatives are consistent with the admission account. Lease's mean read critical section was 1.7–2.0 μs versus Snapshot's 1.1–1.3 μs; the later table-count experiment exposes the per-table leasing work behind this descriptive difference.

H. Does equivalence survive compaction?

Lease and Snapshot remained throughput-equivalent under compaction, but their commit-wait tails differed. The Churn-shaped campaign tested two reader counts, three doses, and compaction on or off over ten rounds. All twelve primary on

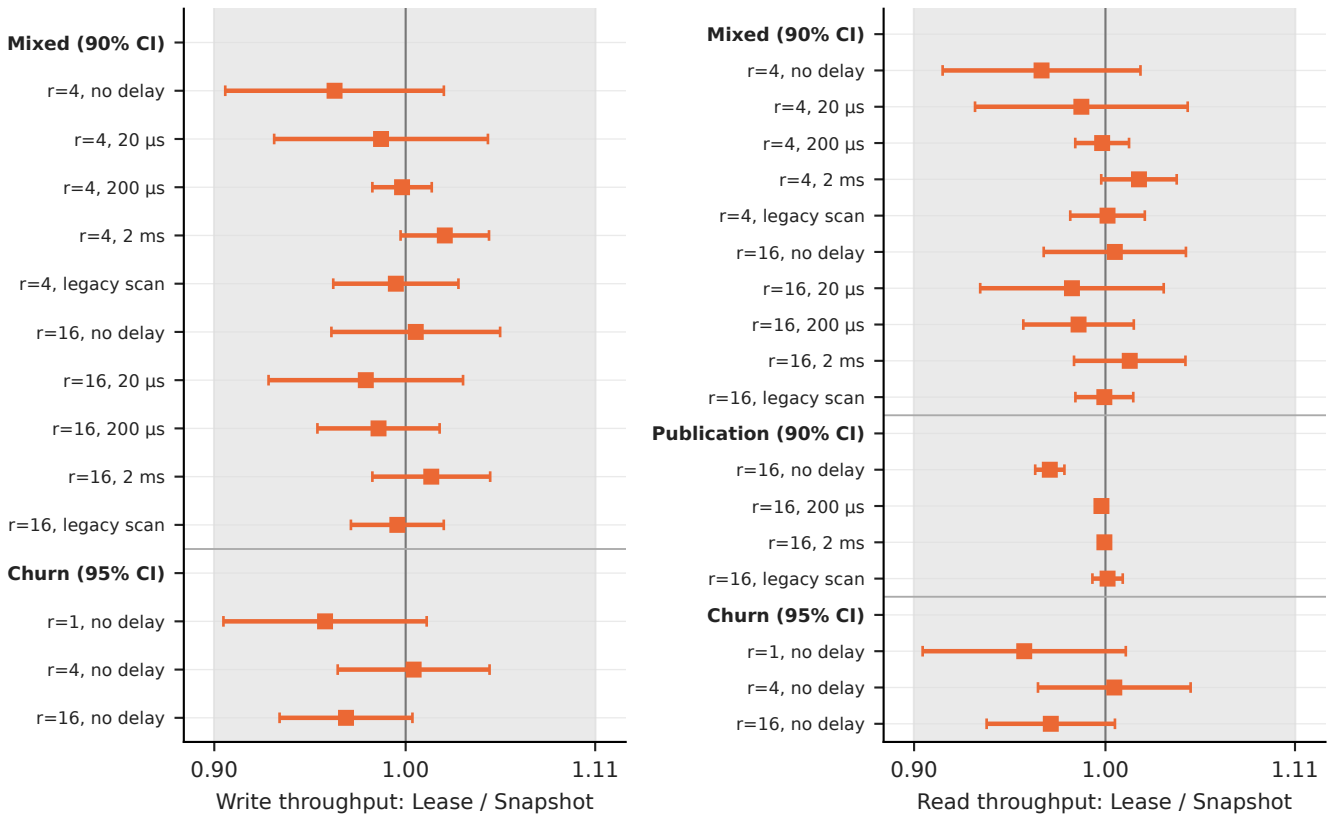


Fig. 6. Lease/Snapshot throughput, ten triplets per cell. Points are paired geometric ratios; bars are 90% paired log-t CIs for Mixed and Publication, and 95% for Churn. Shading shows the rounded $[0.90, 1.11]$ margin. Churn was not an equivalence test.

TABLE X
CHURN, TEN TRIPLETS PER CELL. RATES ARE GEOMETRIC MEANS (OPERATIONS/S). LEASE/SNAPSHOT HAS 95% PAIRED-t AND PERCENTILE BOOTSTRAP CIS. FLUSHES ARE ARITHMETIC MEANS INCLUDING DRAIN, ORDERED COARSELOCK/LEASE/SNAPSHOT.

r	Outcome	CoarseLock	Lease	Snapshot	Ratio [95% t CI]	Bootstrap CI	Flushes
1	W	697	747	781	0.957 [0.905, 1.012]	[0.910, 0.997]	65.3/70.1/73.2
1	R	1,400	807	844	0.956 [0.904, 1.011]	[0.909, 0.997]	–
4	W	585	702	699	1.004 [0.963, 1.047]	[0.970, 1.038]	54.6/65.9/65.5
4	R	4,703	3,089	3,074	1.005 [0.964, 1.048]	[0.970, 1.038]	–
16	W	415	569	588	0.968 [0.933, 1.004]	[0.938, 0.995]	38.8/53.2/55.1
16	R	13,351	10,050	10,357	0.970 [0.937, 1.005]	[0.941, 0.997]	–

tests met H-C's equivalence prediction after the separate write and read Holm corrections; all twelve off reference tests were equivalent too. The closest cell was writes at sixteen readers and 200 μs: Lease/Snapshot was 0.949, 90% CI $[0.909, 0.991]$. It met the margin but allowed a small Lease deficit (Table XI).

Throughput hid a consistent tail difference. Lease's commit-wait p99 was 1.43–7.69 times Snapshot's across all six on cells, with every 95% paired-ratio CI excluding one. Absolute arithmetic means of trial p99 were 21–246 μs for Lease, 12–177 μs for Snapshot, and 306–9221 μs for CoarseLock (Fig. 8). This fits the disclosed input-open asymmetry: CoarseLock and Lease open compaction inputs while holding the exclusive lock, whereas Snapshot opens them outside all locks. Lease's mean start hold was 161–354 μs. We did not intervene on this asymmetry, so it is not an established cause of the tail gap or an inherent cost of leasing.

Retirement ran in all on cells. Mean retirement lags spanned 0.74–1.26 ms for CoarseLock, 0.86–6.21 ms for Lease, and 0.96–3.81 ms for Snapshot. Cell mean retired-table high-water marks were 4.0–5.9 across the designs (Table XII). At low doses, counts were roughly timer-limited, with cell means of 32–36 compactions per trial. At 2 ms they were flush-limited: CoarseLock completed mean counts of 9–10 versus 22.0–22.8 in the alternatives because it produced fewer flushes. These observations do not establish compaction starvation.

CoarseLock's write penalty survived compaction. At zero,

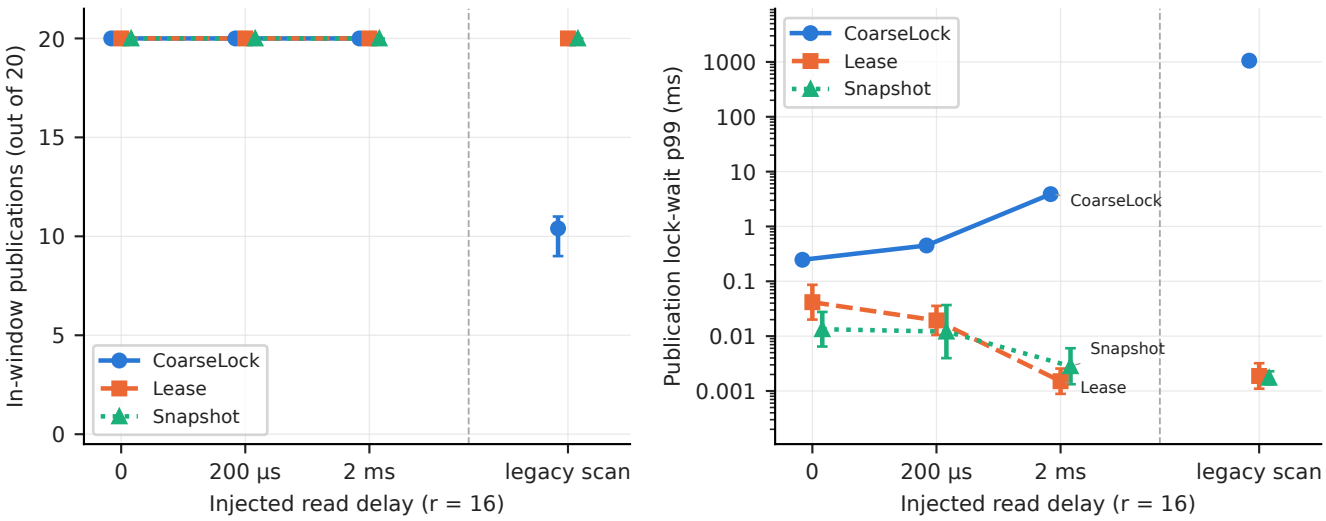


Fig. 7. Publication workload, ten triplets per cell. Left: arithmetic mean in-window count with min–max bars. Right: geometric mean trial publication lock-wait p99 and exponentiated 95% log-t CIs. Wait counters include drain.

200, and 2000 μs, S was 0.919, 0.681, 0.439 at four readers and 0.851, 0.656, 0.393 at sixteen. Turning compaction on increased throughput in every design and cell: write ratios were 1.17–2.66 and read ratios 1.31–2.86. Off cells accumulated a maximum cell mean of 70.0 overlapping tables, while on cell means stayed at 1.0–2.8. The on/off comparison therefore includes relief from read amplification as well as compaction work; it does not isolate compaction's cost.

An unpredicted result further limits the earlier read-gain interpretation. At 2 ms, CoarseLock/Lease read ratios with compaction were 0.912 [0.892, 0.933] at four readers and 0.820 [0.806, 0.835] at sixteen, with 95% CIs. Without compaction, the ratios were 1.154 and 1.081. A post-hoc, untested reading is that off-condition CoarseLock flushes less and carries fewer tables, cell means of 8.0 versus 12.0 final tables at 2 ms. Some earlier read advantage may therefore involve table count as well as admission. We did not separate those contributions.

I. Does equivalence survive more tables?

Lease's throughput equivalence survived more tables only when another part of the workload limited reads. The 480-trial campaign crossed four disjoint table counts, two reader counts, and zero or one writer. Lease leases every active table per read; Snapshot pins one set. H-T1 detected declining Lease/Snapshot read throughput in both read-only families: Page's $L = 295$ and 296, with Holm-adjusted $p = 5.88 \times 10^{-9}$ and 1.60×10^{-9}. It did not detect a decline with a free writer.

All H-T2 predictions held: the applicable outcomes were equivalent at four tables, and read-only throughput was not equivalent at 256 tables for either reader count. Across 4, 16, 64, and 256 tables, read-only ratios were 0.975, 0.956, 0.918, 0.712 at four readers and 0.989, 0.962, 0.851, 0.663 at sixteen (Table XIII). At sixteen readers the deficit was already outside the equivalence margin at 64 tables, ratio 0.851, 90% CI $[0.837, 0.866]$. At 256 it reached 0.663, 90% CI $[0.660, 0.666]$, or about 34% fewer reads. With a free writer, all sixteen applicable read and write tests were equivalent, with point ratios 0.943–1.045. The exclusive commit hold limits reads in that setting and can mask per-read lifetime cost; cells with a free writer could not expose the read-only loss. The Publication cells had no writer but reached only 24 tables; their roughly 3% Lease deficit at zero delay points the same way, though this link is post hoc and untested.

The critical-section measurements locate the increasing work inside Lease's read path. Its read-only mean hold grew from 326 ns to 3.4 μs at four readers and from 393 ns to 8.5 μs at sixteen, while Snapshot stayed below 1 μs. Figure 9 shows throughput beside the separate lifecycle microbenchmark. A benchmark capture/release cycle cost Lease 27–974 ns at one goroutine and 58–1767 ns at sixteen; Snapshot stayed near 21 and 88 ns, respectively. Publication reversed the allocation trade-off: Lease cost 104–128 ns with no allocations, while Snapshot cost 654–3200 ns with three allocations (Table XIV). In these implementations, Snapshot moves table-proportional work from frequent reads to less frequent publication.

The campaign's per-table critical-section increase was roughly 12 ns at four readers and 32 ns at sixteen, compared with about 3.8 ns per table in the single-goroutine microbenchmark. Cold cache lines on real table objects spread through the heap are a possible explanation, not a tested one. The microbenchmark uses synthetic table objects and isolates lifecycle operations; it is not an end-to-end read measurement.

Disjoint ranges are a deliberate limit. A lookup reads blocks from one table, but all designs still pay up to N in-memory range checks, under the manager lock only for CoarseLock. With a fixed keyspace, keys per table shrink as N grows. We therefore attribute neither the entire campaign throughput

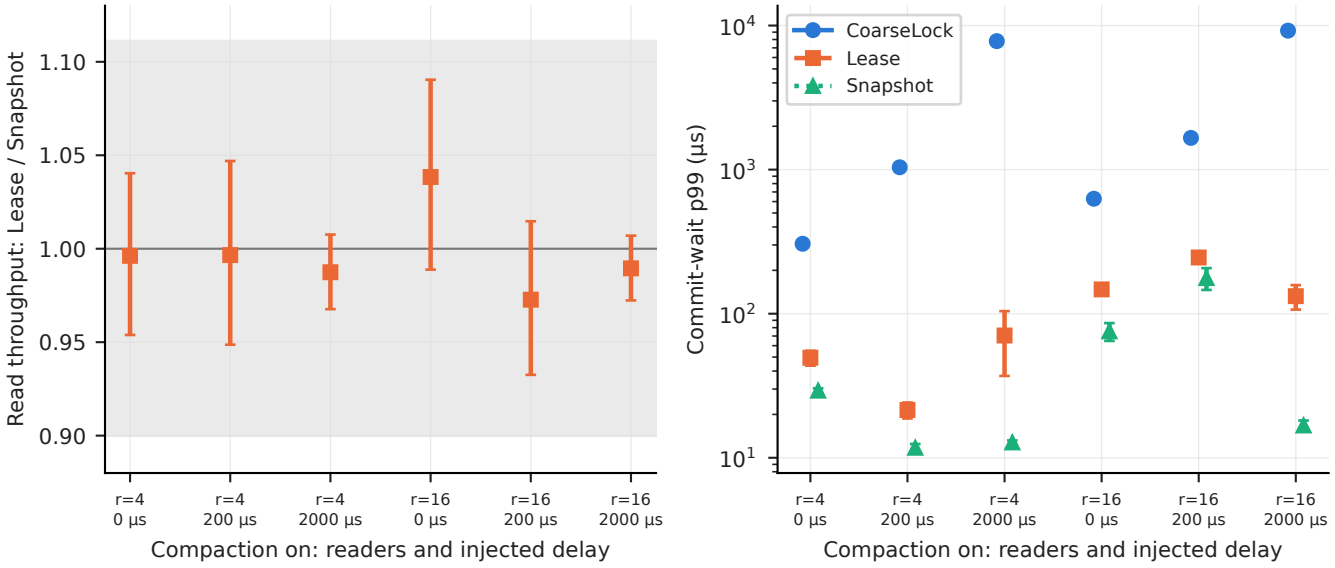


Fig. 8. Compaction-on cells, ten triplets each. Left: Lease/Snapshot read throughput with 90% paired log-t CIs and the equivalence band. Right: arithmetic mean trial commit-wait p99 with 95% t CIs, in μs on a log axis. Ratio intervals and absolute-duration intervals use different summaries.

TABLE XI
COMPACTION-ON THROUGHPUT. LEASE/SNAPSHOT RATIOS HAVE 90% CIS AND HOLM-ADJUSTED CLASSIFICATIONS; S AND COARSELOCK/LEASE READ RATIOS HAVE 95% CIS. ALL RATIOS ARE PAIRED GEOMETRIC MEANS.

		Lease/Snapshot writes		Lease/Snapshot reads		S	$R_{\mathrm{CL}}/R_{\mathrm{L}}$
r	Delay	Ratio [90% CI]	Class	Ratio [90% CI]	Class	[95% CI]	[95% CI]
4	0	0.989 [0.944, 1.035]	equivalent	0.996 [0.954, 1.040]	equivalent	0.919 [0.881, 0.958]	1.494 [1.433, 1.557]
4	200 μs	0.997 [0.949, 1.047]	equivalent	0.997 [0.949, 1.047]	equivalent	0.681 [0.651, 0.712]	1.301 [1.244, 1.361]
4	2 ms	0.995 [0.975, 1.015]	equivalent	0.987 [0.968, 1.008]	equivalent	0.439 [0.430, 0.448]	0.912 [0.892, 0.933]
16	0	1.030 [0.980, 1.083]	equivalent	1.038 [0.989, 1.090]	equivalent	0.851 [0.800, 0.904]	1.408 [1.326, 1.496]
16	200 μs	0.949 [0.909, 0.991]	equivalent	0.973 [0.933, 1.015]	equivalent	0.656 [0.636, 0.676]	1.239 [1.204, 1.276]
16	2 ms	0.998 [0.980, 1.016]	equivalent	0.990 [0.972, 1.007]	equivalent	0.393 [0.386, 0.400]	0.820 [0.806, 0.835]

TABLE XII
COMPACTION-ON LIFECYCLE MEASUREMENTS. ARITHMETIC MEANS OF TRIAL COMMIT-WAIT P99 AND TRIAL MEAN RETIREMENT LAG, INCLUDING DRAIN.

		Commit-wait p99 (μs)			Mean retirement lag (ms)		
r	Delay	CoarseLock	Lease	Snapshot	CoarseLock	Lease	Snapshot
4	0	305.8	49.5	29.2	0.85	0.86	0.96
4	200 μs	1038.3	21.5	11.7	0.78	1.52	1.29
4	2 ms	7795.5	70.6	12.8	0.76	5.11	2.86
16	0	627.9	147.5	75.4	1.26	1.02	0.97
16	200 μs	1660.5	245.6	176.7	0.85	1.73	1.47
16	2 ms	9220.9	132.4	16.8	0.74	6.21	3.81

change to atomics alone nor these ratios to overlapping-table search in a production LSM.

VI. DISCUSSION

For this class of engine, the practical choice depends on measured hold time, admission, and table count. The following implications are bounded by the tested engine and workload.

1) Measure lock-held read duration before replacing a coarse lock. The short-hold cells had at most about 8% point-estimate write loss and higher read throughput; millisecond holds had large losses in both. Those observations are operating points, not a confidence-bounded threshold for another engine.
2) Treat the writer's own exclusive work as a separate problem. Neither Lease nor Snapshot changes conflict validation or WAL fsync under the lock. Moving reader I/O outside the lock leaves that commit floor and delayed validation intact.
3) Include admission policy and exclusive acquisitions per commit in the performance model. The intervention

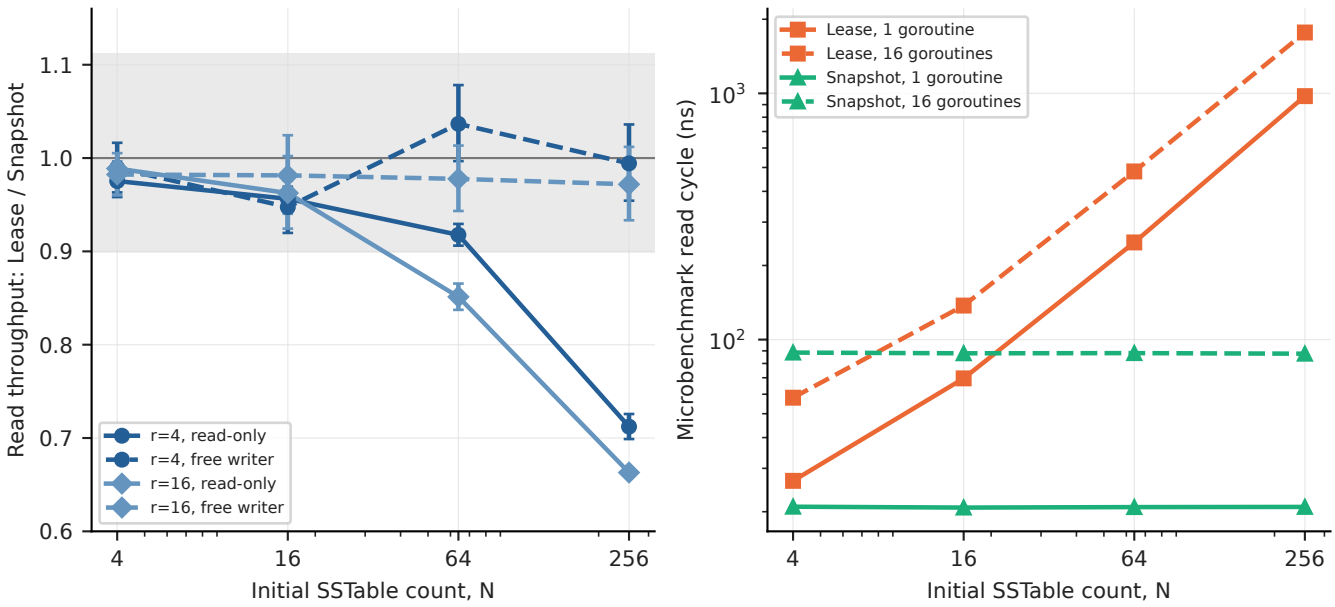


Fig. 9. Table-count campaign and separate lifecycle microbenchmark. Left: Lease/Snapshot read ratios with 90% paired log-t CIs; solid lines are read-only and dashed lines have one free writer. Right: arithmetic means of three benchmark counts, with no campaign CI. The benchmark isolates capture/release and excludes search and I/O.

TABLE XIII
TABLE-COUNT CAMPAIGN, TEN TRIPLETS PER CELL. THROUGHPUT RATIOS ARE LEASE/SNAPSHOT WITH 90% PAIRED LOG-t CIS AND PLANNED HOLM-ADJUSTED CLASSES. CS IS ARITHMETIC MEAN READ CRITICAL SECTION (μS). A DASH MEANS NO WRITER AND NO APPLICABLE WRITE TEST.

			Reads		Writes		Mean read CS (μs)		
N	r	Writers	Ratio [90% CI]	Class	Ratio [90% CI]	Class	CoarseLock	Lease	Snapshot
4	4	0	0.975 [0.958, 0.993]	equivalent	–	–	13.520	0.326	0.256
4	4	1	0.989 [0.963, 1.016]	equivalent	0.988 [0.960, 1.018]	equivalent	13.546	2.274	2.109
4	16	0	0.989 [0.981, 0.997]	equivalent	–	–	40.980	0.393	0.327
4	16	1	0.982 [0.960, 1.005]	equivalent	0.980 [0.953, 1.007]	equivalent	21.682	1.708	1.542
16	4	0	0.956 [0.944, 0.969]	equivalent	–	–	10.524	0.494	0.294
16	4	1	0.947 [0.920, 0.976]	equivalent	0.943 [0.912, 0.974]	equivalent	13.792	2.304	2.095
16	16	0	0.962 [0.924, 1.002]	equivalent	–	–	26.098	0.914	0.610
16	16	1	0.981 [0.940, 1.024]	equivalent	0.977 [0.927, 1.031]	equivalent	19.559	1.768	1.556
64	4	0	0.918 [0.906, 0.929]	equivalent	–	–	10.250	1.016	0.291
64	4	1	1.037 [0.997, 1.078]	equivalent	1.045 [1.001, 1.092]	equivalent	14.962	2.558	2.117
64	16	0	0.851 [0.837, 0.866]	not equivalent	–	–	28.984	2.689	0.802
64	16	1	0.978 [0.943, 1.013]	equivalent	0.979 [0.936, 1.023]	equivalent	20.045	2.175	1.609
256	4	0	0.712 [0.699, 0.726]	not equivalent	–	–	12.157	3.370	0.269
256	4	1	0.994 [0.954, 1.036]	equivalent	1.011 [0.967, 1.057]	equivalent	17.045	3.528	2.184
256	16	0	0.663 [0.660, 0.666]	not equivalent	–	–	33.703	8.512	0.877
256	16	1	0.972 [0.933, 1.012]	equivalent	0.985 [0.938, 1.035]	equivalent	22.349	3.900	1.602

supported the account that a second acquisition offers another reader admission. Removing it improved CoarseLock writes at higher doses but reduced reads and increased read p99 in both designs. It is a trade-off, not a free gain.

4) Lease was sufficient for throughput near four initial tables and when commits limited reads, but its read cost grew with table count. It lost about 15–34% at 64–256 tables in the read-only cells that failed equivalence. Engines with many live files should not expect the four-table equivalence to hold. Snapshot's publication work and the implementation-specific compaction tail gap also belong in the comparison.

A. What we did not test

The remaining questions concern workloads and implementation choices that these campaigns do not isolate. Future work includes overlapping-table search, open-loop and rate-limited loads, fsync outside the exclusive lock with group commit and multiple writers, other reader-writer admission policies, real device reads with larger datasets, and other operating systems. A direct intervention on compaction input opening could test whether removing the disclosed asymmetry closes the commit-tail gap. Each change requires correctness validation as well as performance measurement; the present results do not predict the outcome.

TABLE XIV
SEPARATE LIFECYCLE MICROBENCHMARKS: ARITHMETIC MEAN OF THREE COUNTS, NS/OPERATION. READ-CYCLE BENCHMARKS ALLOCATE NO MEMORY. PUBLICATION ALLOCATION COUNTS ARE PER OPERATION.

	Read cycle, 1 goroutine		Read cycle, 16 goroutines		Publication		Publication allocations	
N	Lease	Snapshot	Lease	Snapshot	Lease	Snapshot	Lease	Snapshot
4	26.7	21.0	58.1	88.6	109.1	653.7	0	3
16	69.6	20.8	137.5	88.0	104.4	717.5	0	3
64	248.1	20.9	481.8	88.1	110.2	1035.3	0	3
256	974.4	20.9	1766.7	87.6	127.7	3200.3	0	3

VII. THREATS TO VALIDITY

A. Construct

Injected delay controls read cost but does not reproduce a device. Every visited point-read block incurs delay, without application block-cache hits; the small dataset was likely resident in the OS page cache. The treatment changes character across levels, spinning below 1 ms and mostly sleeping at 2 ms. Thus the doses are not a uniform intervention scaled only in duration. Sixteen readers exceed twelve logical CPUs, making CPU contention part of those cells.

Closed-loop clients wait for completion before issuing another request and can understate latency relative to an open-loop arrival process [33]. Uniform access, one or zero writers, and the small keyspace further restrict the workload construct. Synchronization and flush counters include drain, unlike throughput. A p99 over twenty publication events is near a maximum; p50 may be unstable in a mixed fast/blocked cycle. Bimodality was inferred without a full reported histogram.

The first two campaigns ran no compaction, which limits their equivalence result. The follow-up closes that gap for its own fixed-timer, merge-all implementation, not a level- or size-based policy. Its on/off comparison also changes table count and read amplification, so it does not isolate compaction overhead. The table-count campaign uses disjoint ranges and shrinking tables; all designs still do range checks. Its microbenchmark uses synthetic table objects and test helpers, not complete storage reads or flushes.

B. Internal

Randomized interleaving and pairing reduce order bias but do not eliminate host effects or measurement bias [30]. The first campaign's CoarseLock and Snapshot end/start read ratios were both about 1.09, with log ratios 0.087 and 0.090, below but near the 0.10 threshold. Each arm/time anchor is a single trial. The later campaigns have no anchors; round trends are descriptive. `platform.json` records the Balanced base plan. The overnight campaigns ran on AC power with other user applications closed by the operator; the Best performance overlay on the Balanced base plan was read back from the registry after both had finished, not during them. This does not independently verify the overlay throughout execution. Defender and other host activity remain possible sources of variation.

WAL fsync under the exclusive lock, without group commit, sets an approximately 1 ms low-dose cycle floor. This is a source-informed interpretation without a sync-off intervention: the optional run was not performed because the harness rejects that setting. The first protocol's statement that Lease and Snapshot do not hold the lock across delay applied to readers only; delayed writer validation was noticed after collection. The exploratory cycle model assumes an unmeasured second wait; the intervention removes that assumption only below threshold. The input-open asymmetry was disclosed before the compaction campaign, but its contribution to commit tails was not tested.

Smoke trials were seen before each freeze and excluded. The first campaign had 21 smoke trials before analysis rules were fixed; the intervention had eight after predictions were written; compaction had fifteen; table count had six. The table-count single-round smoke ratio near 0.66 at 256 tables and sixteen readers pointed in the predicted direction, although predictions were written first. No design value, margin, or prediction changed because of these runs. An operator also stopped the first start at 38 of 519 trials for power-supply reasons unrelated to results. It was discarded without analysis; only status lines were seen. Fixtures were constructed without fsync.

The race detector reported no data races in the engine and campaign harness tests under Linux on WSL2; one older overload test fails under the detector for a timing reason because the slowdown disturbs the overload it expects. The Artifact Appendix identifies that test. This was not a Windows race-detector run, and no race tests or engine trials were rerun for this rewrite.

C. External

One engine, one laptop, one OS, and one file system cannot establish production generality. Production systems may have group commit, different validation paths, block caches, multiple writers, or different lifetime and admission policies. The three designs are not exact implementations of LevelDB, RocksDB, or Pebble. The admission equation is an approximate explanation of this Go lock and workload, not a general readers-writers law. The author was the sole experimenter and analyst, with no independent replication.

D. Statistical

The declared equivalence margin is a judgment, not a production requirement. Holm adjustment applies within the specified families, not across every outcome in the paper.

Churn uses CIs without an equivalence prediction. The intervention has only five paired units, omits Snapshot, and applies no multiplicity correction, including the six P2 comparisons; its precision and strength are limited. The tighter-margin containment count is post hoc and unadjusted. Bootstrap sensitivity checks interval robustness but does not remove the assumptions of paired inference. Repeated measurements on one host estimate within-host variation and are not independent replications across machines or implementations.

The admission explanation was generated after seeing the first campaign, and only its specified follow-up predictions were tested on new data. Compaction-tail and table-count-cache interpretations remain untested. A CI including one does not establish equality, and failing to detect a trend with a writer does not establish absence of a per-table cost. These distinctions matter when interpreting both equivalence and the explanatory model.

VIII. Related Work

LSM research supplies the storage structure and a broader set of concurrency goals than this experiment tests. O'Neil et al. introduce the LSM-tree [1], and Luo and Carey survey LSM-based storage techniques [2]. Golan-Gueta et al. present cLSM, built on LevelDB using non-blocking data structures, with a shared-exclusive lock synchronizing puts against the merge's global pointer updates [7]. FloDB reworks the in-memory component to scale it [8], and Dong et al. review RocksDB's development priorities [6]. These works build or explain scalable structures; we instead measure reader lock hold as a controlled variable in one fixed engine.

Background work presents a different source of latency. SILK schedules internal flush and compaction I/O to prevent client latency spikes [9], while Luo and Carey study performance stability and write stalls [10]. Those stalls arise from background work that our first two campaigns excluded and our later fixed-timer compaction experiment only partly represents. Our on/off result includes changing read amplification and is not a decomposition of background-work cost.

Lifetime protection connects the designs to reclamation methods. Michael's hazard pointers provide safe memory reclamation for lock-free objects [19]. Hart et al. compare quiescent-state, epoch, and hazard-pointer reclamation and find no globally optimal scheme [20]. Lease and Snapshot are file-lifetime analogues in a setting where I/O dominates many cells. The table-count result shows why their per-read and per-publication costs still matter when a writer no longer masks them.

Reader-writer synchronization research studies both scalability and admission. Courtois et al. formulate the readers-writers problem [12]; Mellor-Crummey and Scott study scalable reader-writer synchronization [13]; Brandenburg and Anderson address phase-fair synchronization [14]. Calciu et al. develop NUMA-aware reader-writer locks [15], Liu et al. study passive locks for read-mostly synchronization [16], and Dice and Kogan's BRAVO adds a biased reader path to an existing lock [17]. These works focus on lock scalability and blocking bounds. Our finding concerns how admission couples reader and writer throughput inside a storage engine, including the effect of two exclusive acquisitions for one logical commit.

Logging work bears on the writer hold that our reader designs leave intact. DeWitt et al. describe group commit [21]; Aether uses early lock release and flush pipelining [22]. We do not evaluate those techniques. Their relevance is that an engine with a different logging path may have a different commit floor and therefore expose different lifetime costs.

Evaluation methods inform how much can be concluded from these measurements. Georges et al., Hoefler and Belli, Traeger et al., and Mytkowicz et al. address rigorous evaluation and measurement pitfalls [29]–[32]. Schroeder et al. distinguish open and closed loads [33]; YCSB supplies a common workload generator that we did not use [34]. Lakens explains equivalence testing [27], and Nosek et al. motivate pre-registration as a separation between prediction and observation [28]. Our paired design, disclosed margin, frozen predictions, and separate exploratory interpretations follow those concerns without making the single-host study an independent replication.

IX. Conclusion

In this engine, CoarseLock's point-estimate write loss stayed below about 8% at measured read holds below about 40 μs, but reached 24–29% around 170 μs and 64–69% around 1.9 ms, where reads also fell. A separate intervention supported the admission explanation for the unexpected short-read gain: the second exclusive acquisition offered another reader admission, and removing it traded reader throughput and tail latency for CoarseLock writes. Lease and Snapshot met the declared throughput margin near four initial tables, including under compaction, while publication wait and compaction commit tails required different conclusions. Without a writer, Lease's per-table work broke equivalence at large counts, reaching 34% fewer reads at 256 tables and sixteen readers. The four campaigns therefore establish conditional evidence on one engine and host with closed-loop load and fsync under the exclusive lock, rather than a general ranking of lifetime designs.

Acknowledgment

Anthropic's Claude and OpenAI's Codex were used to assist with engineering, analysis scripts and drafting of text. The author reviewed the results and is responsible for the content.

Artifact Availability

Source, protocols, and analysis and manuscript generators are in https://github.com/itsramhere/selfdbms, branch `research/x-readcost`. Trial records and analysis outputs for all four campaigns are committed on that branch. The claim-to-evidence map is `paper/CLAIM_EVIDENCE_MAP_X.md`. The generators copy frozen estimates and CIs, compute descriptive arithmetic, and emit the tables and prose macros; no engine trial or inferential analysis of campaign data was rerun for the rewrite.

TABLE XV
PAPER NAMES AND REPOSITORY IDENTIFIERS.

Repository identifier	Paper name
A	CoarseLock
AMIN, A-min	Lease
B	Snapshot
X1	Mixed workload
X2	Publication workload
X3	Churn workload
second	two-acquisition commit (default)
inline	one-acquisition commit
legacy_scan	legacy scan
X-intervention-v1	the intervention
X-compaction-v1	the compaction campaign
X-tablecount-v1	the table-count campaign

The first campaign used revision c53ee5b, plan SHA-256 prefix 68ad96f878c1, and runner prefix 22060ffc4885. The intervention used 4c2749b, plan 2e44fcacb3e7, and runner 5dc57e7d85d7. The compaction campaign used c342832, plan 5330cbddf01b, and runner 3a05470d307b. The table-count campaign used adf754b, plan 0bc0042b8b8a, and runner 050c3210e9e8. Protocols are research/experiments/phase_i7/X_PREREGISTRATION.md, research/experiments/phase_i7/X_INTERVENTION_PREREGISTRATION.md, research/experiments/phase_i7/X_COMPACTION_PREREGISTRATION.md, and research/experiments/phase_i7/X_TABLECOUNT_PREREGISTRATION.md.

Master seeds were 20261030, 20261103, 20261104, and 20261105, respectively. Paired seeds are the first eight SHA-256 bytes of decimal master, cell or shared base identifier, and round separated by vertical bars, interpreted as a big-endian signed 64-bit value; compaction on/off shares the base identifier. Plans and ledgers retain execution order, retry and approval bindings, and the excluded smoke runs; the first start stopped after 38 successful trials is disclosed in the protocol. The older timing-sensitive race-detector failure is the E4 overload test; the engine and X harness tests reported no races under Linux on WSL2 with Go 1.27.3.

REFERENCES

[1] P. O'Neil, E. Cheng, D. Gawlick, and E. O'Neil, "The log-structured merge-tree (LSM-tree)," *Acta Informatica*, vol. 33, no. 4, pp. 351–385, 1996.

[2] C. Luo and M. J. Carey, "LSM-based storage techniques: a survey," *The VLDB Journal*, vol. 29, no. 1, pp. 393–418, 2020.

[3] S. Ghemawat and J. Dean, "LevelDB." [Online]. Available: https://github.com/google/leveldb

[4] Meta Platforms, "RocksDB." [Online]. Available: https://github.com/facebook/rocksdb

[5] Cockroach Labs, "Pebble." [Online]. Available: https://github.com/cockroachdb/pebble

[6] S. Dong, A. Kryczka, Y. Jin, and M. Stumm, "RocksDB: Evolution of development priorities in a key-value store serving large-scale applications," *ACM Trans. Storage*, vol. 17, no. 4, Art. 26, 2021.

[7] G. Golan-Gueta, E. Bortnikov, E. Hillel, and I. Keidar, "Scaling concurrent log-structured data stores," in *Proc. EuroSys*, 2015, Art. 32.

[8] O. Balmau, R. Guerraoui, V. Trigonakis, and I. Zablotchi, "FloDB: Unlocking memory in persistent key-value stores," in *Proc. EuroSys*, 2017, pp. 80–94.

[9] O. Balmau, F. Dinu, W. Zwaenepoel, K. Gupta, R. Chandhiramoorthi, and D. Didona, "SILK: Preventing latency spikes in log-structured merge key-value stores," in *Proc. USENIX ATC*, 2019, pp. 753–766.

[10] C. Luo and M. J. Carey, "On performance stability in LSM-based storage systems," *Proc. VLDB Endow.*, vol. 13, no. 4, pp. 449–462, 2019.

[11] W. Pugh, "Skip lists: A probabilistic alternative to balanced trees," *Commun. ACM*, vol. 33, no. 6, pp. 668–676, 1990.

[12] P. J. Courtois, F. Heymans, and D. L. Parnas, "Concurrent control with 'readers' and 'writers'," *Commun. ACM*, vol. 14, no. 10, pp. 667–668, 1971.

[13] J. M. Mellor-Crummey and M. L. Scott, "Scalable reader-writer synchronization for shared-memory multiprocessors," in *Proc. PPoPP*, 1991, pp. 106–113.

[14] B. B. Brandenburg and J. H. Anderson, "Reader-writer synchronization for shared-memory multiprocessor real-time systems," in *Proc. ECRTS*, 2009, pp. 184–193.

[15] I. Calciu, D. Dice, Y. Lev, V. Luchangco, V. J. Marathe, and N. Shavit, "NUMA-aware reader-writer locks," in *Proc. PPoPP*, 2013, pp. 157–166.

[16] R. Liu, H. Zhang, and H. Chen, "Scalable read-mostly synchronization using passive reader-writer locks," in *Proc. USENIX ATC*, 2014, pp. 219–230.

[17] D. Dice and A. Kogan, "BRAVO: Biased locking for reader-writer locks," in *Proc. USENIX ATC*, 2019, pp. 315–328.

[18] The Go Authors, "sync package: RWMutex." [Online]. Available: https://pkg.go.dev/sync#RWMutex

[19] M. M. Michael, "Hazard pointers: Safe memory reclamation for lock-free objects," *IEEE Trans. Parallel Distrib. Syst.*, vol. 15, no. 6, pp. 491–504, 2004.

[20] T. E. Hart, P. E. McKenney, A. D. Brown, and J. Walpole, "Performance of memory reclamation for lockless synchronization," *J. Parallel Distrib. Comput.*, vol. 67, no. 12, pp. 1270–1285, 2007.

[21] D. J. DeWitt, R. H. Katz, F. Olken, L. D. Shapiro, M. R. Stonebraker, and D. A. Wood, "Implementation techniques for main memory database systems," in *Proc. ACM SIGMOD*, 1984, pp. 1–8.

[22] R. Johnson, I. Pandis, R. Stoica, M. Athanassoulis, and A. Ailamaki, "Aether: A scalable approach to logging," *Proc. VLDB Endow.*, vol. 3, no. 1–2, pp. 681–692, 2010.

[23] E. B. Page, "Ordered hypotheses for multiple treatments: A significance test for linear ranks," *J. Amer. Statist. Assoc.*, vol. 58, no. 301, pp. 216–230, 1963.

[24] D. J. Schuirmann, "A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability," *J. Pharmacokinet. Biopharm.*, vol. 15, no. 6, pp. 657–680, 1987.

[25] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.

[26] B. Efron, "Bootstrap methods: Another look at the jackknife," *Ann. Statist.*, vol. 7, no. 1, pp. 1–26, 1979.

[27] D. Lakens, "Equivalence tests: A practical primer for t tests, correlations, and meta-analyses," *Soc. Psychol. Personal. Sci.*, vol. 8, no. 4, pp. 355–362, 2017.

[28] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor, "The preregistration revolution," *Proc. Natl. Acad. Sci. USA*, vol. 115, no. 11, pp. 2600–2606, 2018.

[29] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. OOPSLA*, 2007, pp. 57–76.

[30] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!," in *Proc. ASPLOS*, 2009, pp. 265–276.

[31] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems," in *Proc. SC*, 2015, pp. 1–12.

[32] A. Traeger, E. Zadok, N. Joukov, and C. P. Wright, "A nine year study of file system and storage benchmarking," *ACM Trans. Storage*, vol. 4, no. 2, Art. 5, 2008.

[33] B. Schroeder, A. Wierman, and M. Harchol-Balter, "Open versus closed: A cautionary tale," in *Proc. NSDI*, 2006.

[34] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, "Benchmarking cloud serving systems with YCSB," in *Proc. ACM SoCC*, 2010, pp. 143–154.